

Template

jueves, 9 de abril de 2026 12:36

- **Template Method** (metodo plantilla)
- Es un patrón de diseño de **comportamiento**. Define el **esqueleto de un algoritmo** en una superclase pero permite que las subclases cambien algunos pasos.

El template method llama a dos tipos de operaciones:

- **operaciones primitivas** (abstractas en AbstractClass y que las subclases tienen que definir)
- **operaciones concretas** definidas en AbstractClass y que las subclases pueden redefinir si hace falta (hook methods)

Ejemplo

-Hace una bebida caliente

1) hervir agua

2) prepara ingrediente

3) servir en taza

4) Agregar extras

Pero :

Para un te -> agregar limón

Para un café -> agregas azúcar

Cuando usarlo?

Cuando varios algoritmos tiene la misma estructura pero cambian en algunos pasos

Ejemplo en código

```
abstract class Bebida {  
    final void preparar() {  
        hervirAgua();  
        prepararIngrediente();  
        servir();  
        agregarExtras();  
    }  
  
    abstract void prepararIngrediente();  
    abstract void agregarExtras();  
  
    void hervirAgua() { }  
    void servir() { }  
}
```

Creacionales → Se enfocan en la instanciación de objetos.

Factory Method decide qué objeto crear.

Estructurales → Se enfocan en la composición y organización de clases/objetos.

Adapter conecta dos clases incompatibles.

De comportamiento → Se enfocan en la colaboración y comunicación entre objetos.

State : El objeto delega su comportamiento a clases que representan cada estado.

1 La Clase Abstracta declara métodos que actúan como pasos de un algoritmo, así como el propio método plantilla que invoca estos métodos en un orden específico. Los pasos pueden declararse abstractos o contar con una implementación por defecto.

2 Las Clases Concretas pueden sobrescribir todos los pasos, pero no el propio método plantilla.

